

VLSI Architecture of Full-Search Variable-Block-Size Motion Estimation for HEVC Video Encoding

Niras Cheeckottu Vayalil¹ ✉, Yinan Kong¹

¹Department of Engineering, Macquarie University, Sydney 2109, NSW, Australia

✉ E-mail: niras.cheeckottu-vayalil@mq.edu.au

ISSN 1751-858X

Received on 14th July 2016

Revised 12th February 2017

Accepted on 20th February 2017

E-First on 24th April 2017

doi: 10.1049/iet-cds.2016.0267

www.ietdl.org

Abstract: Motion estimation (ME) is the most computationally intensive task in video encoding. This study proposes a full-search variable-block-size ME for the high-efficiency video coding or H.265 specification. The proposed method reduces memory requirements to a large extent by following a Morton order for data reading and a sum of absolute differences reuse strategy. The data bandwidth demand is also diminished by broadcasting data into multiple processing elements. This ME accelerator supports variable-block-size prediction blocks ranging from 8×4 to 64×64 , and is reconfigurable in various search ranges for a trade-off between performance and area. The proposed method for very-large-scale integration (VLSI) architecture is synthesized with 32 nm technology, and is capable of real-time encoding of ultra-high-definition (4K-UHD, at 30 Hz) video with a search range of 64 pixels in both horizontal and vertical directions, operating at a frequency of 282 MHz.

1 Introduction

The latest video compression standard developed by the Joint Collaborative Team on Video Coding (JCT-VC) is known as high-efficiency video coding (HEVC/H.265) [1]. HEVC/H.265 achieves significantly better coding efficiency than its predecessor, Advanced video coding (AVC/H.264) [2], by incorporating a larger block size and a variety of partitioning modes. In video encoding, motion estimation (ME) is the most computationally intensive task with $\sim 80\%$ of the total computational load [3]. The computational complexity increases manifold for ME in HEVC/H.265 as compared with the previous standard, since the basic encoding block, the coding tree unit (CTU), size is increased from 16×16 to 64×64 (16 times as many pixels as the former standard) and by introducing a hierarchical quad-tree structure. On the other hand, the demand for ultra-high-definition (UHD) with a resolution of 3840×2160 (4K-UHD) introduces another four times increase in computations for ME.

Several algorithms exist for AVC/H.264 to reduce the number of search points for ME such as diamond search (DS) [4], new three-step search [5] and enhanced predictive zonal search [6]. Many of them assume a unimodal error surface with a global minimum, but that model is not true, in general, for videos, whereas many other fast-search algorithms are not suitable for hardware implementation due to the algorithm complexity, a sequential processing requirement or the algorithm dependency on thresholds. A hardware-oriented modified DS is proposed in [7] to mitigate the above problems. Another search-point reduction algorithm for hardware implementation is proposed in [8]. A low-complexity variable-block-size ME is proposed in [9], utilising a fast-search algorithm based on a 9-point block concept. As a result of the quad-tree partitioning structure and other complexities, various ME algorithms proposed for AVC/H.264 as in [9–11] are no longer suitable for its successor HEVC/H.265 [12].

Owing to the variety of partitioning modes and the quad-tree structure present in HEVC/H.265, the mode decision and coding unit (CU) size selection involve enormous computing time if it tries all available CUs and selects the best. CU decision or mode decision algorithms are proposed in [13–17] to decide the best CU, and some algorithms reduce the computational complexity by up to 50% in experimental cases. An adaptive-search-window algorithm presented in [18] tries to reduce the search-window size based on the motion vector (MV) and the cost value of previous blocks. ME is suitable for parallelisation, which leads to a parallel framework

for the central processing unit + graphics processing unit platforms [19–22]. Several algorithm-level optimisations are recommended in [23–25] including suggestions of early termination. However, most of these proposals are for software implementations and many of them are not attractive for hardware realisation. Moreover, software implementation in video generally consume more power and are not suitable for real-time encoding.

To meet the high computational requirements for HEVC/H.265, this paper proposes a hardware architecture for full-search ME. Although the fast-search algorithm requires less hardware resources [26], full-search ME always comes up with the best results as it covers all the search points of the fast-search method. The proposed architecture utilises the Morton order for data reading as well as the sum of absolute differences (SADs) [27] reuse strategy for finding MVs in the hierarchical quad-tree coding structure of HEVC/H.265. SAD reuse is a crucial technique to achieve real-time processing in hardware, where computed SAD values are stored and hierarchically used for larger blocks.

A straightforward implementation of variable-block-size ME is to use a $P \times Q$ size SAD computation unit similar to [28] or [29], and a search range of $L \times L$ pixels which generates $L \times L$ SADs of $P \times Q$ blocks. For the ME in a video with a frame rate f Hz and a frame size of $W \times H$ pixels, the aforementioned hardware requires a bandwidth of $(W/P \times H/Q) \times (L \times L) \times (P \times Q) \times f$ bytes per second. The architecture introduced in this paper saves bandwidth by broadcasting reference frame (RF) pixels to multiple processing elements (PEs) or SAD computation units, for both horizontal and vertical directions. The architecture has an $L \times A$ PE array, which has an input of $L + A - 1$ pixels in a row, and computes a row of L different SADs of an $A \times A$ block in each clock cycle, thus requiring L clock cycles to complete a search-window of $L \times L$ pixels. Hence, the bandwidth requirement is $(W/A \times H/A) \times (L + A - 1) \times (L + A - 1) \times f$ bytes per second, providing an approximate bandwidth saving of A^2 times over the conventional methods, as $L \gg A$. L and A are taken as 64 and 8, respectively, reducing the bandwidth by a factor of 52.

Since a set of SADs is generated from each rows of PE array, the output pattern results in groups of varying x coordinates, which is not most suited for hierarchical SAD computation [30]. This issue is addressed in [30] by saving all SAD data into a random access memory (RAM), reading the data and reorganising them. However, this demands at least 512 kB RAM to save all 8×8 block's SADs inside a 64×64 CTU and for streaming data

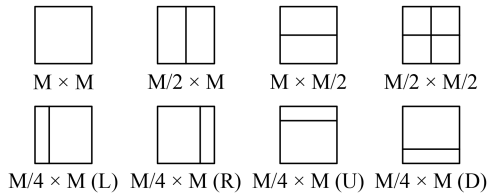


Fig. 1 Partitioning of CB into PB in HEVC

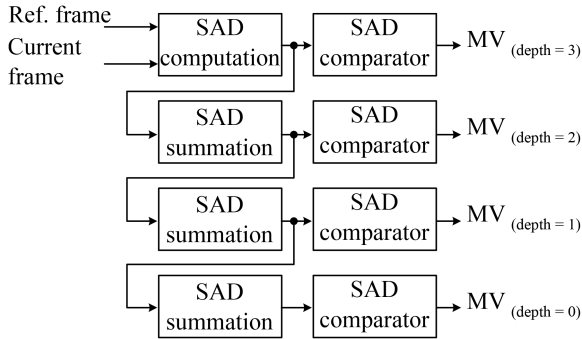


Fig. 2 ME hardware architecture. RAMs are used for storing partial and SAD results in each summation stage

continuously. The proposed architecture uses Morton order for data reading, thus it does not need to save all SAD data, instead it is sufficient to save L rows of L SADs in each hierarchical level. To facilitate computation of $M \times M/2$ and $M/2 \times M$ type prediction block (PB) SADs, some additional memory is required in each stage, thus a total of 36 kB RAM is used by the proposed architecture.

2 Full-search variable-block-size ME in HEVC

The HEVC/H.265 standard introduces a very flexible and hierarchical coding structure. A picture is divided into CTUs of $L \times L$ where L takes the values of 16, 32 or 64, depending on the encoders, for different memory and computational requirements [1]. All CTUs have the same size, and the CTU replaces a fixed 16×16 macroblock in the previous standard. The support for larger block sizes provides higher coding efficiency, ~50% bit-rate savings in HEVC/H.265 compared with its predecessor for an equivalent reproduction quality [2], and is also beneficial for encoding higher-resolution videos such as UHD. Each CTU consists of $L \times L$ samples of a luma coding tree block (CTB) and the corresponding chroma CTBs of $L/2 \times L/2$ samples. The luma and chroma CTBs can be split recursively into square coding block (CB) in 32×32 , 16×16 down to 8×8 pixels (luma samples) in a quad-tree structure.

One luma CB and the associated chroma CB form a CU. A CB can be split differently to PBs and transform blocks. The CB is split non-recursively into the PB as shown in Fig. 1, where the lower four partition types are known as asymmetric motion partitioning (AMP). The prediction mode for a CU maybe either intra or inter depending on whether it uses intra-picture prediction or inter-picture prediction, respectively. In intra-picture prediction mode, only $M \times M$ and $M/2 \times M/2$ partition types are allowed, whereas in inter-picture prediction mode the AMP cannot be used if M is < 16 for luma or AMP is disabled. In the inter-picture prediction mode, the size 4×4 is not allowed to reduce memory bandwidth. The luma PB with associated chroma PBs forms a prediction unit (PU).

The objective of the ME is to find the best matching block for all kinds of PU types of a current frame (CF) from a previous or future frames (RFs) that gives minimal residues. In an area of picture frame that has many details the smaller block size gives minimal residues. On the other hand, for the smooth areas larger block size gives better compression. The ME is usually done in a predefined search window, because it is expected that motion is confined to that area for near or consecutive frames, and generally the search-window size increases with increasing resolution and

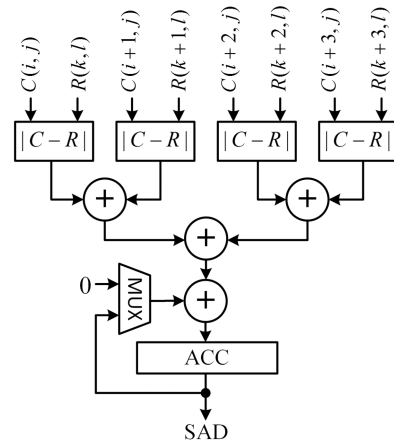


Fig. 3 PE for calculating SAD of 4 pixels

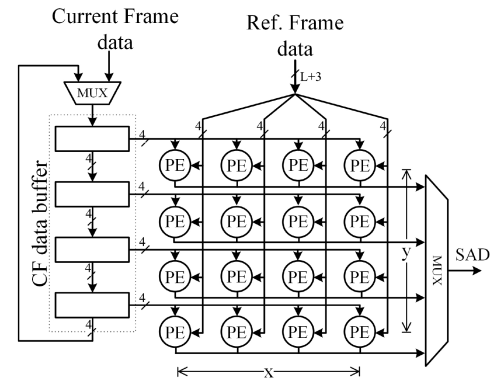


Fig. 4 PE array for ME SAD computation. The $L \times 4$ PE array can be reconfigurable in the x-direction; in the above $L = 4$

decreases with increasing frame rates. The ME should be done for all the PU types (variable-block-size ME) to get the best coding efficiency.

The SAD is one error measure criterion and is widely used because of its simplicity. The SAD of an $M \times N$ block can be calculated using (1)

$$\text{SAD} = \sum_{i=0}^{M-1} \sum_{j=0}^{N-1} |C(i, j) - R(i, j)| \quad (1)$$

where C and R represent current block and reference block pixels, respectively.

3 Hardware architecture of HEVC/H.265 variable-block-size ME

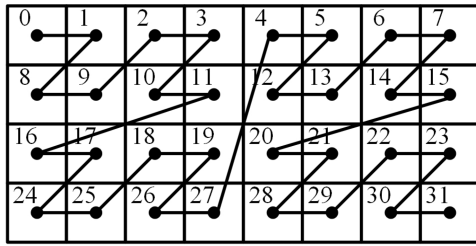
The proposed architecture for HEVC/H.265 variable-block-size ME uses a full-search method. The schematic representation of the proposed architecture is given in Fig. 2 and consists of three main blocks, namely SAD computation, SAD summation and SAD comparator. The SAD computation block finds SAD values with the help of PE, the SAD summation block calculates various SAD values of different kinds of blocks in the HEVC/H.265 quad-tree structure, and the comparator blocks find the minimum SAD values and thus MVs of these blocks.

3.1 SAD computation

The heart of the SAD computation is a PE as given in Fig. 3, which calculates the SAD values of 4 pixels of the CF and the corresponding RF. Each PE accumulates a 4×4 pixel SAD. The minimum block size according to the HEVC/H.265 specification is 4×4 pixels, so that the PEs are arranged as an array of $L \times 4$ as shown in Fig. 4, where L is shown as 4 for simplicity, which is configurable in the x-direction. In each cycle, a row of RF data ($L + 3$ pixels) is broadcast into PE elements, whereas the CF data is

Table 1 Detailed data flow in the PE array with $L = 4$

Cycle		PE($x = 0$)	PE($x = 1$)	PE($x = 2$)	PE($x = 3$)	SAD
0	PE($y = 0$)	$C(0:3, 0), R(0:3, 0)$	$C(0:3, 0), R(1:4, 0)$	$C(0:3, 0), R(2:5, 0)$	$C(0:3, 0), R(3:6, 0)$	
	PE($y = 1$)	—	—	—	—	
	PE($y = 2$)	—	—	—	—	
	PE($y = 3$)	—	—	—	—	
1	PE($y = 0$)	$C(0:3, 1), R(0:3, 1)$	$C(0:3, 1), R(1:4, 1)$	$C(0:3, 1), R(2:5, 1)$	$C(0:3, 1), R(3:6, 1)$	
	PE($y = 1$)	$C(0:3, 0), R(0:3, 1)$	$C(0:3, 0), R(1:4, 1)$	$C(0:3, 0), R(2:5, 1)$	$C(0:3, 0), R(3:6, 1)$	
	PE($y = 2$)	—	—	—	—	
	PE($y = 3$)	—	—	—	—	
2	PE($y = 0$)	$C(0:3, 2), R(0:3, 2)$	$C(0:3, 2), R(1:4, 2)$	$C(0:3, 2), R(2:5, 2)$	$C(0:3, 2), R(3:6, 2)$	
	PE($y = 1$)	$C(0:3, 1), R(0:3, 2)$	$C(0:3, 1), R(1:4, 2)$	$C(0:3, 1), R(2:5, 2)$	$C(0:3, 1), R(3:6, 2)$	
	PE($y = 2$)	$C(0:3, 0), R(0:3, 2)$	$C(0:3, 0), R(1:4, 2)$	$C(0:3, 0), R(2:5, 2)$	$C(0:3, 0), R(3:6, 2)$	
	PE($y = 3$)	—	—	—	—	
3	PE($y = 0$)	$C(0:3, 3), R(0:3, 3)$	$C(0:3, 3), R(1:4, 3)$	$C(0:3, 3), R(2:5, 3)$	$C(0:3, 3), R(3:6, 3)$	rdy
	PE($y = 1$)	$C(0:3, 2), R(0:3, 3)$	$C(0:3, 2), R(1:4, 3)$	$C(0:3, 2), R(2:5, 3)$	$C(0:3, 2), R(3:6, 3)$	
	PE($y = 2$)	$C(0:3, 1), R(0:3, 3)$	$C(0:3, 1), R(1:4, 3)$	$C(0:3, 1), R(2:5, 3)$	$C(0:3, 1), R(3:6, 3)$	
	PE($y = 3$)	$C(0:3, 0), R(0:3, 3)$	$C(0:3, 0), R(1:4, 3)$	$C(0:3, 0), R(2:5, 3)$	$C(0:3, 0), R(3:6, 3)$	
4	PE($y = 0$)	$C(0:3, 0), R(0:3, 4)$	$C(0:3, 0), R(1:4, 4)$	$C(0:3, 0), R(2:5, 4)$	$C(0:3, 0), R(3:6, 4)$	rdy
	PE($y = 1$)	$C(0:3, 3), R(0:3, 4)$	$C(0:3, 3), R(1:4, 4)$	$C(0:3, 3), R(2:5, 4)$	$C(0:3, 3), R(3:6, 4)$	
	PE($y = 2$)	$C(0:3, 2), R(0:3, 4)$	$C(0:3, 2), R(1:4, 4)$	$C(0:3, 2), R(2:5, 4)$	$C(0:3, 2), R(3:6, 4)$	
	PE($y = 3$)	$C(0:3, 1), R(0:3, 4)$	$C(0:3, 1), R(1:4, 4)$	$C(0:3, 1), R(2:5, 4)$	$C(0:3, 1), R(3:6, 4)$	
:	:	:	:	:	:	:

**Fig. 5** Morton order or Z-order method for loading data into CF data buffer; 32×64 pixels are shown, where each square represents a block of 8×8 pixels

stored in local memory along with the PE array (CF data buffer). One important difference of this arrangement from many other structures including [30, 31] is that the proposed structure does not use any multiplexers in the reference data lines, hence halves the number of reference data lines. Although the mux with double RF data lines increases throughput, most of the time one half of the RF-lines are idle, which reduces hardware efficiency.

The detailed data flow of this arrangement is given in Table 1, where C and R represent CF and RF pixels, respectively. The PEs are arranged as a two-dimensional (2D) array of size $L \times 4$, and each PE computes the SAD of a row of four CF pixels and RF pixels, and accumulate them in the Accumulator register as shown in Fig. 3.

In every four clock cycles, PE starts to accumulate new absolute differences by with the help of Multiplexer (MUX) selection. Each PE accumulates the SAD of 4×4 blocks in different x, y search positions. As an example consider PE at $x = 1$ and $y = 0$. As shown in Table 1, in cycle 0 the PE finds the SAD of four current pixels of columns 0–3 (or $x = 0$ to 3) of row 0, denoted as $C(0:3, 0)$, and four reference pixels of columns 1–4 of row 0, denoted as $R(1:4, 0)$. In the next cycle, PE adds the SADs of $C(0:3, 1)$ and $R(1:4, 1)$ to the accumulator. Hence, in cycle 3 this PE has SAD of a 4×4 block $C(0:3, 0:3)$ and $R(1:4, 0:3)$. Note that the reference data are broadcasted to PEs, and the PEs are finding SADs of different positions; thus, in cycle 4 the PEs in row 0 start new accumulation and row 1 has the SADs of x from 0 to $L - 1$ and $y = 1$.

After an initial latency of four clocks, the PE array generates L SAD values of $4 \times L$ pixels in each clock cycle. In other words, the PE array requires N clocks to cover a search area of $N \times (L + 3)$ pixels. As an example, if the search area is $128 \times L$ pixels, the RF

line moves one row in the y -direction in each clock, and 132 clock cycles (because of initial latency) are required to search an area of $128 \times (L + 3)$ pixels. After completion of scanning in the y -direction, the RF-lines start to broadcast the next set (advancing $L + 3$ pixels in the x -direction) of reference data, thus RF-lines loses its continuity, and the SAD computation has again some latency, but still negligible i.e. 4 in 128 clocks (four out of the search range of the y -direction). The CF data circulates in the CF buffer until scanning in the y -direction completes, and then loads a new set of data. Since only one row of SAD values are ready in each clock cycle, 4–1 muxes are used to select a row and route it to the SAD summation block. The description is given with $4 \times L$ PEs as an example, but the actual hardware uses four sets of $4 \times L$ PEs, where $L = 64$, and it can compute 8×4 , 4×8 and 8×8 SAD values in parallel.

3.2 SAD summation

The architecture is designed to stream data through SAD computation, SAD comparator and the SAD comparator blocks to generate MVs. To achieve this, CF data are loaded in a Z-order or Morton order [32], i.e. after finding all SADs of an 8×8 CF block, the next 8×8 load into the CF buffer memory from a 64×64 block following Morton order as shown in Fig. 5. This allows the design to free the memories associated with lower-level HEVC quad-tree blocks of the SAD computation as soon as those block computations are complete. This reduces the large memory requirements and the associated bandwidths, and only a few kilobytes of RAM are required in each summation stage, and that can be easily integrated inside the chip.

The SAD summation block with the SAD comparators are detailed in Fig. 6. Three such units are required to compute SAD values for block hierarchies of up to 64×64 in size. The figure depicts computation of 8×16 , 16×8 and 16×16 block-size SADs from the 8×8 SADs. The working of this module is as follows: when the SAD computation block finds SAD values for block '0' in Fig. 5, it saves them in RAM 'A'. Next, the SAD computation finds SADs of block '1' and these are saved in RAM 'B', and simultaneously finds the first 16×8 SAD values by summing with the previously saved SAD of block '0'. Similarly while receiving block '8' (next block in Morton order) SADs, 8×16 block SAD values are calculated and overwrite RAM 'A' with block '8' SAD values (block '0' values are no longer required). In this way, the SAD summation finds all SAD values of the 8×16 and 16×8 blocks. The 16×16 SAD can be found by

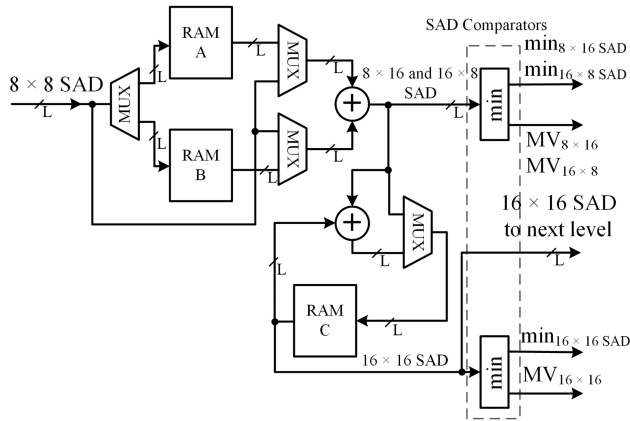


Fig. 6 SAD summation block with the SAD comparators. This calculates SAD values for higher levels using SAD values from the lower level in HEVC quad-tree SADs, and finds the minimum of these SAD values. The above shows calculation of 8×16 , 16×8 and 16×16 block SAD values and MVs from 8×8 block SAD values

summing up either from 8×16 or from 16×8 blocks. RAM 'C' and one adder in the SAD summation block are used for this purpose. The first 16×8 or 8×16 block SADs are saved in RAM 'C' and then added with the next 8×16 or 16×8 to get SADs of 16×16 blocks. The subsequent blocks can be clocked at a half rate by saving 16×16 SAD results in RAM 'C' again and reading at half clock rate. This improves power efficiency as well as critical path timing, because during additions the bit-width of the data path increases, thus higher-level modules need more time for computation than lower-level modules.

3.3 SAD comparator

The SAD comparator consists of a comparator tree of $\log_2(L)$ stages to find the minimum of L SAD values, where each comparator finds the lowest value of two binary inputs and gives it as a result, and also indicates the position of the lowest. The proposed architecture's critical path lies in the comparator tree used for finding minimum of the $L8 \times 8$ SADs obtained from the SAD computation step, where L has been set as 64, and thus the critical path consists of a total of six 14 bit comparators. Note that the 8×8 SAD has 14 bit wide, as the bit-width increases in each addition unless truncating. The comparator trees are also used after each SAD summation block. In each cycle from a row of SAD data, the comparator finds the minimum of these and then compares it with the previous row minimum. Hence, when the hardware completes the search in the y -direction the comparator has the SAD minimum of that area and the MVs.

4 Results and discussion

A first-order comparison of designs is given in Table 2; the most common proposals for full-search ME are usually a variation of the structure given in [12].

Less number of comparators in the design [30] is obvious as it has fewer kinds of supported PB types. The bandwidth decreases, and the area increases in proportion to A^2 , approximately. Most importantly, the critical path of the proposed design is six comparators, whereas it could be the memory accessing in [30]. The table gives the minimum memory requirement for a highly parallelised version H64 P8 [30], but such structure is big enough for many field programmable gate arrays (FPGAs), for instance, it cannot be implemented in XC5VLX330T [30], and the less parallelised versions require memory many times of this.

The proposed architecture is coded in Very High Speed Integrated Circuit (VHSIC) Hardware Description Language (VHDL) and simulated and verified thoroughly using ModelSim simulator. A snapshot of the simulation window is given in Fig. 7, showing the MVs and SAD values of different blocks when hardware completes ME for the first CTU of the test video sequence ICE (4CIF). The proposed architecture is configured for a

Table 2 Comparison of space-time complexities of different integer ME hardware

	EL '13 [12]	ICIP '14 H64 P8 [30]	This work
Area			
adders	8708 ^a	58,304 ^a	$LA^2/4 \times 7 = 7552$
comparators	920 ^a	256 ^a	$8L = 512$
memory, kB	20	128 ^a	$3L^2 = 32$
bandwidth	B^a	$B/232$	$B \left(\frac{L+A-1}{L \cdot A} \right)^2 = B/52$
required ^b			
critical path	tree adders for 16 numbers ^a	six comparators or memory access ^a	$\log_2(L) =$ six comparators

^aEstimated from the given information.

^b $B = W \cdot H \cdot L^2 \cdot f$.

search height of 64 pixels and a search width of 64 pixels. Although many other configurations are possible, this provides a good trade-off between device area (or resource utilisation) and speed. This requires 72 clock cycles for a 8×8 block, and $72 \times 64 = 4608$ clocks are required to complete a search for one 64×64 size CTU, for a search area of 64×64 pixels. Since the data bandwidth is comparatively low and data fetch is in a predetermined order, it is assumed that data can be fed directly from the external RAM where the picture frames are stored. The proposed hardware does not support AMP, and experimental studies demonstrate that AMP only improves the coding efficiency by 0.8% with the computational increase of 14% [33].

4.1 FPGA synthesis results

The proposed design is compiled using Xilinx ISE design suite for a XC5VLX330T FPGA. Although the proposed design minimises the RAM requirement, it needs a few and that is resolved with FPGA slices. The synthesis report shows that the design can operate with a maximum clock frequency of 84.96 MHz. Table 3 shows a comparison with [30] implemented in the same FPGA. There are three proposals in [30], namely H64 P2, H64 P4 and H64 P8, and the H64 P8 design has the best performance among them even though it does not fit in this Xilinx Virtex-5 FPGA. Thus, our design is compared with the H64 P8 design of [30]. A basic difference between our proposed architecture and [30] is that the latter saves all first SAD computation results in memory then reorganises using a transposer, and feed to SAD comparators and 4-1 adders for calculating other higher-level SADs in the quad-tree structure. Whereas the proposed method utilises the Morton order for reading data as well as a unique RAM arrangement, and its control structure facilitates data flowing in a streamline fashion through the hardware, resulting in high throughput. From this table, it is clear that the proposed design uses fewer resources, with similar frame rates and search area.

Hardware architectures for SAD computations are proposed in [29, 35], which compute whole 64×64 CTU SADs at single search position within 372.2 and 167.7 ns, thus to finish ME in an entire 64×64 search range these demands 1.525 ms 686.9 μ s, respectively. Hence, these cannot be used for real-time encoding of 4K-UHD (3840×2160) video as it has 2040 CTUs of size 64×64 pixels. The architecture proposed in this paper broadcasts or shares the same reference data lines in both x and y directions as shown in Fig. 4, where $L+3$ reference data lines are used by $L \times 4$ PEs, efficiently utilising data loaded to the hardware and hence speeding up overall computations.

Another full-search integer ME architecture is proposed in [34], and implemented in Xilinx Virtex-5 and Virtex-7 FPGAs. The comparison is given in Table 3, where the frame rate with 4K-UHD is an estimated value from Virtex-5 operating frequency as it requires 4096 cycles neglecting initial delays, for the search range of 64×64 pixels. The design is essentially a 2D array of 64×64 absolute difference units followed by an SAD adder-tree block. The design can compute different block-size SAD values in an

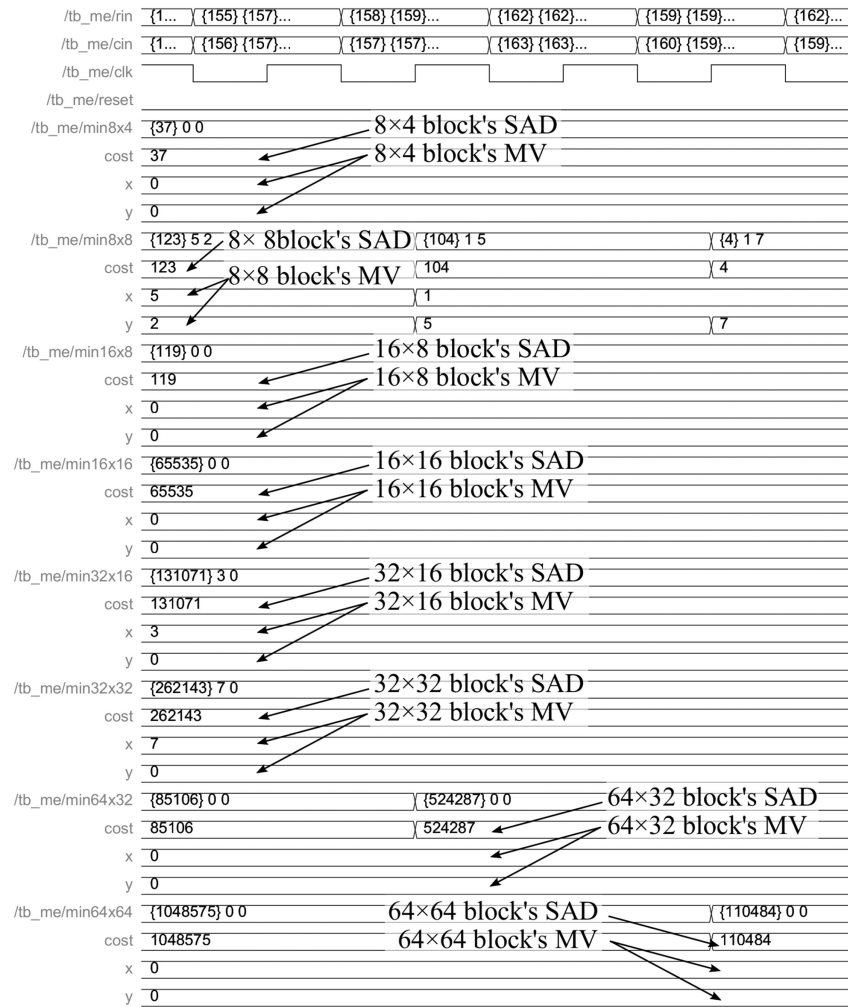


Fig. 7 Simulation window of ModelSim simulator, simulated with ICE (4CIF) test video sequence. The snapshot shows MVs and SAD values of the first CTU of the video when hardware completes motion search with search range of ± 8 pixels

Table 3 Comparison of the proposed design with other HEVC/H.265 full-search integer ME implementation in Xilinx Virtex-5 devices

	ICIP '14 H64 P8 [30]	JRTIP '16 [34]	ICCE '16 [28]	This work
block size	8×8 – 64×64 (four kinds)	8×4 – 64×64 (27 kinds)	4×4 – 64×64	8×4 – 64×64 (15 kinds)
search range	64×64	64×64	64×64	64×64
maximum supported resolution	4K-UHD(2160 p at 13.4 Hz)	4K-UHD(2160 p at 19 Hz ^a)	HD(720 p at 6 Hz ft ^a)	4K-UHD(2160 p at 9 Hz)
operating frequency, MHz	125	159	190.785	84.96
LUTs	2,09,434	1,84,288	17,992 ^b	1,53,314
flip-flops	1,99,066	1,78,620	8841 ft ^b	36,368
Block RAM	9.57 MB	36 kB	nil	nil

^aEstimated using operating frequency of the design.

^bSAD architecture section only.

entire CTU block in a single clock cycle for a search position. A 2D shift register array is used for storing current and reference block data, and the shift registers for reference block data need to shift the data in up, down and right directions, resulting in a complex arrangement. Although the design performs better in terms of throughput, it consumes 29% more Look-up Tables (LUTs) and five times as many flip-flops than the architecture proposed in this paper. As discussed earlier, these hardware need ~ 64 times more bandwidth than the proposed architecture.

4.2 Application-specific integrated circuit (ASIC) synthesis

The proposed architecture is compiled using the Synopsys Design Compiler (version K-2015.06) in topographical mode, which gives the best correlation with physical design results, using the Synopsys Armenia educational department (SAED) 32 nm digital

standard-cell library. This proposed ME accelerator requires a clock frequency of 282.009 MHz for encoding 4K-UHD (3840×2160 p at 30 Hz) videos in real time. Thus, the design is constrained to 330 MHz while compiling in Synopsys. The proposed design met this constraint with operating conditions 1.16 V and worst-case temperature 125°C. The critical path, lies in comparator tree after the 8×8 SAD computation block, has a delay of 2.98 μ s. A comparison of various hardware proposals is shown in Table 4, where the gate count of the proposed architecture is estimated from the synthesised area compared with the standard 2-input NAND gate.

The result shows that the proposed design, operating with slightly higher frequency, is able to search the same area with the same video resolution as [12]. The design requires 36 kB RAM and is realised using standard logic cells, included with the gate count given for comparison. The proposed design has 17% less gate

Table 4 Comparison of the proposed design with previous HEVC/H.265 full-search integer ME in ASIC

Design	EL '13 [12]	JSSC '14 [36]	This work
process	65 nm	40 nm	32 nm
block size	8 × 4–64 × 64 (27 kinds, support AMP)	4 × 4–16 × 16 (all AVC)	8 × 4–64 × 64 (15 kinds, without AMP)
search range	64 × 64	±211 H ±106 V	64 × 64
maximum supported definition	2160 p at 30 Hz	4320 p at 48 Hz	2160 p at 30 Hz
operating frequency, MHz	250	210	282.009
gate count, k	3560	2458	2878
memory, kB	20	552	nil ^a
area	—	15.52 mm ²	7.3 mm ²
throughput	250 M pixels/s	1.59 G pixels/s	250 M pixels/s
clocks required/CTU	4105	—	4608
video standard	H.265/HEVC	H.264/AVC	H.265/HEVC

^aIncluded in the gate count.

count and 20 kB less RAM than the previous architecture in the literature for full-search ME for HEVC/H.265. The proposed hardware has significantly less gate count if the gate count for RAM is considered, even though a direct comparison with [36] is not fair, because the AVC/H.264 is much simpler than its successor.

5 Conclusion

This paper proposes a new architecture for full-search variable-block-size ME in the HEVC/H.265 specification. The proposed architecture minimises the usage of RAM by incorporating a data streaming and SAD reuse strategy hierarchically. With a similar clock rate, this architecture has 17% less gate count and 20 kB less RAM than previously proposed full-search ME accelerator architectures for HEVC/H.265. The proposed architecture reduces the bandwidth requirement by a factor of ~64 compared with the other full-search ME for HEVC/H.265. The architecture is designed with 32 nm SAED education design kit and can encode up to 4K-UHD videos in real time.

6 References

- [1] Sullivan, G., Ohm, J., Han, W.-J., *et al.*: 'Overview of the high efficiency video coding (HEVC) standard', *IEEE Trans. Circuits Syst. Video Technol.*, 2012, **22**, (12), pp. 1649–1668
- [2] Ohm, J., Sullivan, G., Schwarz, H., *et al.*: 'Comparison of the coding efficiency of video coding standards – including high efficiency video coding (HEVC)', *IEEE Trans. Circuits Syst. Video Technol.*, 2012, **22**, (12), pp. 1669–1684
- [3] Chen, Z., Xu, J., He, Y., *et al.*: 'Fast integer-pel and fractional-pel motion estimation for H.264/AVC', *J. Vis. Commun. Image Represent.*, 2006, **17**, (2), pp. 264–290, Introduction: Special Issue on emerging H.264/AVC video coding standard
- [4] Tham, J.Y., Ranganath, S., Ranganath, M., *et al.*: 'A novel unrestricted center biased diamond search algorithm for block motion estimation', *IEEE Trans. Circuits Syst. Video Technol.*, 1998, **8**, (4), pp. 369–377
- [5] Li, R., Zeng, B., Liou, M.L.: 'A new three-step search algorithm for block motion estimation', *IEEE Trans. Circuits Syst. Video Technol.*, 1994, **4**, (4), pp. 438–442
- [6] Tourapis, A.M.: 'Enhanced predictive zonal search for single and multiple frame motion estimation', *SPIE Vis. Commun. Image Process.*, 2002, **4671**, pp. 1069–1079
- [7] Ndili, O., Ogunfunmi, T.: 'Algorithm and architecture co-design of hardware-oriented, modified diamond search for fast motion estimation in H.264/AVC', *IEEE Trans. Circuits Syst. Video Technol.*, 2011, **21**, (9), pp. 1214–1227
- [8] Tsai, A.C., Bharanitharan, K., Wang, J.F., *et al.*: 'Effective search point reduction algorithm and its VLSI design for HDTV H.264/AVC variable block size motion estimation', *IEEE Trans. Circuits Syst. Video Technol.*, 2012, **22**, (7), pp. 981–988
- [9] Hsia, S.C., Hong, P.Y.: 'Very large scale integration (VLSI) implementation of low complexity variable block size motion estimation for H.264/AVC coding', *IET Circuits Devices Syst.*, 2010, **4**, (5), pp. 414–424
- [10] Jou, S.Y., Chang, S.J., Chang, T.S.: 'Fast motion estimation algorithm and design for real time QFHD high efficiency video coding', *IEEE Trans. Circuits Syst. Video Technol.*, 2015, **25**, (9), pp. 1533–1544
- [11] Kao, C.Y., Lin, Y.L.: 'A memory-efficient and highly parallel architecture for variable block size integer motion estimation in H.264/AVC', *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, 2010, **18**, (6), pp. 866–874
- [12] Byun, J., Jung, Y., Kim, J.: 'Design of integer motion estimator of HEVC for asymmetric motion-partitioning mode and 4K-UHD', *Electron. Lett.*, 2013, **49**, (18), pp. 1142–1143
- [13] Podder, P.K., Paul, M., Murshed, M.: 'Efficient coding strategy for HEVC performance improvement by exploiting motion features'. 2015 IEEE Int. Conf. on Acoustics, Speech and Signal Processing (ICASSP), April 2015, pp. 1414–1418
- [14] Shen, L., Liu, Z., Zhang, X., *et al.*: 'An effective CU size decision method for HEVC encoders', *IEEE Trans. Multimed.*, 2013, **15**, (2), pp. 465–470
- [15] Shen, L., Zhang, Z., Liu, Z.: 'Adaptive inter-mode decision for HEVC jointly utilizing inter-level and spatiotemporal correlations', *IEEE Trans. Circuits Syst. Video Technol.*, 2014, **24**, (10), pp. 1709–1722
- [16] Xiong, J., Li, H., Meng, F., *et al.*: 'Fast HEVC inter CU decision based on latent sad estimation', *IEEE Trans. Multimed.*, 2015, **17**, (12), pp. 2147–2159
- [17] Zhang, J., Li, B., Li, H.: 'An efficient fast mode decision method for inter prediction in HEVC', *IEEE Trans. Circuits Syst. Video Technol.*, 2015, **PP**, (99), p. 1
- [18] Medhat, A., Shalaby, A., Sayed, M.S., *et al.*: 'Adaptive low complexity motion estimation algorithm for high efficiency video coding encoder', *IET Image Process.*, 2016, **10**, (6), pp. 438–447
- [19] Luo, F., Ma, S., Ma, J., *et al.*: 'Multiple layer parallel motion estimation on GPU for high efficiency video coding (HEVC)'. 2015 IEEE Int. Symp. on Circuits and Systems (ISCAS), May 2015, pp. 1122–1125
- [20] Radicke, S., Hahn, J.U., Wang, Q., *et al.*: 'Bi-predictive motion estimation for HEVC on a graphics processing unit (GPU)', *IEEE Trans. Consum. Electron.*, 2014, **60**, (4), pp. 728–736
- [21] Radicke, S., Hahn, J.U., Wang, Q., *et al.*: 'A parallel HEVC intra prediction algorithm for heterogeneous CPU + GPU platforms', *IEEE Trans. Broadcast.*, 2016, **62**, (1), pp. 103–119
- [22] Xiao, W., Li, B., Xu, J., *et al.*: 'HEVC encoding optimization using multicore CPUs and GPUs', *IEEE Trans. Circuits Syst. Video Technol.*, 2015, **25**, (11), pp. 1830–1843
- [23] Hu, N., Yang, E.H.: 'Fast motion estimation based on confidence interval', *IEEE Trans. Circuits Syst. Video Technol.*, 2014, **24**, (8), pp. 1310–1322
- [24] Yang, S.H., Jiang, J.Z., Yang, H.J.: 'Fast motion estimation for HEVC with directional search', *Electron. Lett.*, 2014, **50**, (9), pp. 673–675
- [25] Zupancic, I., Blasi, S.G., Izquierdo, E.: 'Multiple early termination for fast HEVC coding of UHD content'. 2015 IEEE Int. Conf. on Acoustics, Speech and Signal Processing (ICASSP), April 2015, pp. 1419–1423
- [26] Pastuszak, G., Trochimiuk, M.: 'Algorithm and architecture design of the motion estimation for the H.265/HEVC 4K-UHD encoder', *J. Real-Time Image Process.*, 2016, **12**, (2), pp. 517–529
- [27] Vayalil, N.C., Safari, A., Kong, Y.: 'ASIC design in residue number system for calculating minimum sum of absolute differences'. 2015 Tenth Int. Conf. on Computer Engineering Systems (ICCES), December 2015, pp. 129–132
- [28] Dinh, V.N., Phuong, H.A., Duc, D.V., *et al.*: 'High speed SAD architecture for variable block size motion estimation in HEVC encoder'. 2016 IEEE Sixth Int. Conf. on Communications and Electronics (ICCE), July 2016, pp. 195–198
- [29] Nalluri, P., Alves, L.N., Navarro, A.: 'A novel SAD architecture for variable block size motion estimation in HEVC video coding'. 2013 Int. Symp. on System on Chip (SoC), October 2013, pp. 1–4
- [30] D'huys, T.P.K.C., Momcilovic, S., Pratas, F., *et al.*: 'Reconfigurable data flow engine for HEVC motion estimation'. IEEE Int. Conf. on Image Processing (ICIP), August 2014
- [31] Chen, C., Chien, S., Huang, Y., *et al.*: 'Analysis and architecture design of variable block-size motion estimation for H.264/AVC', *IEEE Trans. Circuits Syst. I, Regul. Pap.*, 2006, **53**, (3), pp. 578–593
- [32] Samet, H.: 'The design and analysis of spatial data structures' (Addison-Wesley, Reading, MA, 1990)
- [33] Kim, I.K., Lee, S., Cheon, M.S., *et al.*: 'Coding efficiency improvement of HEVC using asymmetric motion partitioning'. 2012 IEEE Int. Symp. on Broadband Multimedia Systems and Broadcasting (BMSB), June 2012, pp. 1–4
- [34] Alcocer, E., Gutierrez, R., Lopez-Granado, O., *et al.*: 'Design and implementation of an efficient hardware integer motion estimator for an HEVC video encoder', *J. Real-Time Image Process.*, 2016, pp. 1–11, DOI: 10.1007/s11554-016-0572-4
- [35] Ding, L.F., Chen, W.Y., Tsung, P.K., *et al.*: 'A 212 M pixels 4096 × 2160 p multiview video encoder chip for 3D/quad full HDTV applications', *IEEE J. Solid-State Circuits*, 2010, **45**, (1), pp. 46–58
- [36] Zhou, D., Zhou, J., He, G., *et al.*: 'A 1.59 G pixel/s motion estimation processor with –211 to +211 search range for UHD TV video encoder', *IEEE J. Solid-State Circuits*, 2014, **49**, (4), pp. 827–837